CÓDIGOCLARO · RUTA PYTHON

Tema 08 · Comprensiones y generadores

Guía de estudio · DAM/DAW

Comprensiones, yield, iteradores y evaluación perezosa.

1. Explicación

Una comprensión construye una colección a partir de un iterable y puede incluir una condición. Un generador produce valores gradualmente: una expresión con paréntesis o una función con yield devuelve un iterador, no una lista ya calculada. Esto puede reducir memoria cuando se recorren muchos elementos, pero un generador normalmente se consume una vez. Elige la forma más clara para el tamaño y uso real de los datos.

2. Ejemplo

Código · python

```python
def cuadrados_pares(numeros):
    for numero in numeros:
        if numero % 2 == 0:
            yield numero * numero

print(list(cuadrados_pares([1, 2, 4])))
```

Resultado esperado

```
[4, 16]
```

3. Errores frecuentes

- Esperar que yield devuelva una lista completa de inmediato.
- Intentar recorrer dos veces un generador ya consumido.
- Escribir una comprensión tan anidada que resulte más difícil de leer que un bucle.

4. Ejercicio

Produce los cuadrados de los números positivos de una secuencia usando un generador.

Casos que debe superar

Entrada	Resultado esperado
[]	[]
[-2, 0]	[]
[-1, 2, 3]	[4, 9]

5. Solución explicada

Código · python

```python
def cuadrados_positivos(numeros):
    for numero in numeros:
        if numero > 0:
            yield numero ** 2
```

Criterio de aceptación: Se acepta una comprensión convertida a iterador si los valores se producen en el orden de entrada.

Otras soluciones válidas

- Usar una expresión generadora en vez de una función con yield.
- Usar una comprensión de lista si se necesita guardar todos los resultados.

6. Fuentes oficiales

Python Software Foundation — Data Structures · Python Software Foundation — Generators